

IT3012: Intelligent Agents - Practical 04

Informed Search: Completed Exercise

Student ID: IT24103430 (matching the existing lab filenames)

Part 1: Practical Implementation

Step 1.1: Heuristic functions

`SearchAgent.manhattan_distance(pos, goal)` returns $\text{abs}(x1 - x2) + \text{abs}(y1 - y2)$. `SearchAgent.euclidean_distance(pos, goal)` returns $\text{math.sqrt}((x1 - x2)^2 + (y1 - y2)^2)$.

Checkpoint for start (0, 0), goal (3, 4): Manhattan = 7; Euclidean = 5.0. Both functions are implemented in `agent.py` and verified by `test_suite.py` and `lab04_demo.py`.

Step 1.2: A* search

`astar_search(start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')` uses heapq entries (`f_cost`, `g_cost`, `current_pos`, `path_taken`), where $f = g + h$. The starting `g` is zero. Each legal Up, Down, Left, or Right move increases `g` by one. Walls and grid boundaries are excluded. A `reached_states` set prevents repeated expansions, and `best_cost` records the lowest discovered `g` for each position, suppressing duplicate and stale routes. Both provided heuristics are consistent for this movement model, so expanded states can be closed permanently.

The method returns a list of movement actions on success, `[]` if already at a valid goal, and `None` if no route exists or an endpoint is invalid. An unsupported heuristic raises `ValueError`.

Step 1.3: Decision loop and visual integration

`SearchAgent` defaults to `active_algo = 'AStar'`. The decision loop selects the nearest food by Manhattan distance, calls the selected search algorithm, stores the plan, and executes one action per percept. It tries other food items if the closest one is unreachable. BFS, DFS, and UCS remain available. The heuristic is selected with `agent.heuristic_type`.

The percept uses `all_food` for coordinates and `remaining_food` for the number of pellets. The agent tracks position from the known (0, 0) start because the previous lab hides `agent_pos`. `GridGameGUI` is configured to instantiate `SearchAgent`. Food is collected automatically on entry in the existing environment.

Verification and observations

Command: `python3 -m unittest -v test_suite`. Result: 9 tests passed, including previous labs, both distance checkpoints, A* path legality and optimality against BFS on 30 seeded mazes with both heuristics, unreachable and blocked goals, an already-reached goal, invalid heuristic selection, unreachable nearest food, and full agent/environment integration.

Command: `python3 lab04_demo.py`. On a 20 x 20 open grid from (0, 0) to (10, 0), all four searches return a 10-move path. BFS and UCS each expand 65 nodes; A* with Manhattan or Euclidean each expands 10. Counts exclude the goal because it is returned before successor expansion. This illustrates one case; A* does not always achieve this reduction, particularly when many states tie on `f`.

The deterministic 6 x 6 simulation has a wall at (0, 1), food at (0, 2), (3, 0), and (5, 5), and no hazards or opponents. A* collects all three pellets in 16 steps for a score of 60. The GUI hookup was checked in code; no GUI window was run because the available Python lacks Tkinter. The same environment was tested without a GUI.

Part 2: Theoretical Evaluation

1. What is the key difference between how UCS and A* prioritize nodes?

UCS selects the frontier node with the smallest accumulated path cost $g(n)$. A* selects the node with the smallest $f(n) = g(n) + h(n)$, combining the cost already paid with an estimate of the remaining cost. The heuristic directs exploration toward the goal. If $h(n)$ is zero everywhere, A* reduces to UCS.

2. Why is Manhattan distance admissible in this four-way grid? What if the heuristic is not admissible?

An admissible heuristic never overestimates the true minimum remaining path cost. Each unit-cost horizontal or vertical move can reduce Manhattan distance by at most one. Reaching the goal therefore requires at least $\text{abs}(dx) + \text{abs}(dy)$ moves. Walls can force additional moves but cannot make that lower bound too large. Manhattan distance is also consistent: $h(n) \leq 1 + h(\text{neighbor})$ for every legal adjacent step.

If a heuristic overestimates, A* may delay the optimal route and return a more expensive goal path first; optimality is no longer guaranteed. For graph search that permanently closes expanded states, consistency is additionally required for the standard guarantee. An admissible but inconsistent heuristic generally requires reopening states when a cheaper route is discovered. This implementation closes states because both supplied heuristics are consistent.

3. Is Manhattan distance admissible for eight-way movement? Which metric should replace it?

It depends on diagonal movement cost. If a diagonal costs one, moving from (0, 0) to (1, 1) costs one, but Manhattan estimates two, so it is not admissible. Use Chebyshev distance $\max(\text{abs}(dx), \text{abs}(dy))$ when all eight moves cost one. Euclidean distance also overestimates this unit-cost diagonal example and is not an admissible replacement there.

If orthogonal moves cost one and diagonal moves cost $\sqrt{2}$, Manhattan still overestimates. Euclidean distance is admissible, and octile distance $\max(\text{abs}(dx), \text{abs}(dy)) + (\sqrt{2} - 1) * \min(\text{abs}(dx), \text{abs}(dy))$ is a tighter bound for the eight-way grid. If diagonal moves cost two, Manhattan remains admissible. The heuristic must match the actual movement costs.

4. Propose a stronger heuristic for collecting all remaining food.

Represent each search state as (agent_position, remaining_food_set), with the goal reached when the food set is empty. For nonempty food set F , use $h(\text{position}, F) = \min(\text{distance}(\text{position}, \text{food}) \text{ for food in } F) + \text{MST_cost}(F)$, and use zero for an empty set. MST_cost is the cost of a minimum spanning tree connecting all remaining food items.

Use Manhattan distances as inexpensive edge weights, or precomputed shortest-path maze distances for a tighter wall-aware bound. Any complete collection route must reach a first pellet, costing at least the nearest-food distance. Its remaining journey connects every pellet and costs at least the MST weight. The sum is therefore an admissible lower bound and is at least as strong as nearest-food distance alone. Cache the MST by the remaining food set to reduce repeated computation.

This is a proposed multi-food search heuristic. The implemented exercise plans to one pellet at a time, so its sequence of locally shortest paths does not guarantee the globally shortest route to collect all food. Hidden traps and moving opponents also fall outside the static unit-cost path model.